UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Title

Clinical Appointment and Healthcare Management System

Submitted by:

Team Members

NAME	SRN
Arkul Prathamesh Shenoy	PES1UG23CS103
Arjun MS	PES1UG23CS102
Aryan Arun Hiremath	PES1UG23CS109
Atharv Rawal	PES1UG23CS119

6 - B

Prof. Vikram Dunga

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)

1. Problem Statement: Clinical Appointment and Healthcare Management System

Modern healthcare facilities frequently struggle with fragmented, decentralized, and manual administrative processes. The reliance on disparate systems for patient scheduling, medical record keeping, and billing leads to critical operational inefficiencies. Common issues include overlapping appointment schedules, delayed access to Electronic Health Records (EHR), insecure data management, and bottlenecks in administrative workflows.

To resolve these architectural and operational challenges, this project proposes a centralized "Clinical Appointment and Healthcare Management System." Built strictly on Object-Oriented Analysis and Design (OOAD) principles and the Model-View-Controller (MVC) architectural pattern, this software solution automates healthcare operations. It provides a secure, reliable, and highly scalable platform to streamline the interactions between patients, medical staff, receptionists, and system administrators, ultimately improving clinical workflow efficiency and patient care.

2. Key Features

The system is structurally divided into four core, decoupled modules, each responsible for specific business capabilities:

I. Authentication & Access Control

- **Role-Based Access Control (RBAC):** Secure routing and session management tailored for Patients, Doctors, and Administrators.
- **Staff Management:** Administrative tools to provision, update, and manage hospital staff accounts securely.
- *Architectural Note:* Utilizes the **Factory Design Pattern** to dynamically instantiate appropriate user objects based on their permission roles.

II. Patient Management & Electronic Health Records (EHR)

- **Centralized Patient Profiles:** Complete Create, Read, Update, and Delete (CRUD) operations for managing patient demographics and contact details.
- **Medical History Tracking:** Secure management of clinical consultations, diagnoses, and medical histories.
- *Architectural Note:* Implements the **Singleton Design Pattern** to manage heavy database connection resources efficiently during intensive data retrieval.

III. Core Appointment Engine

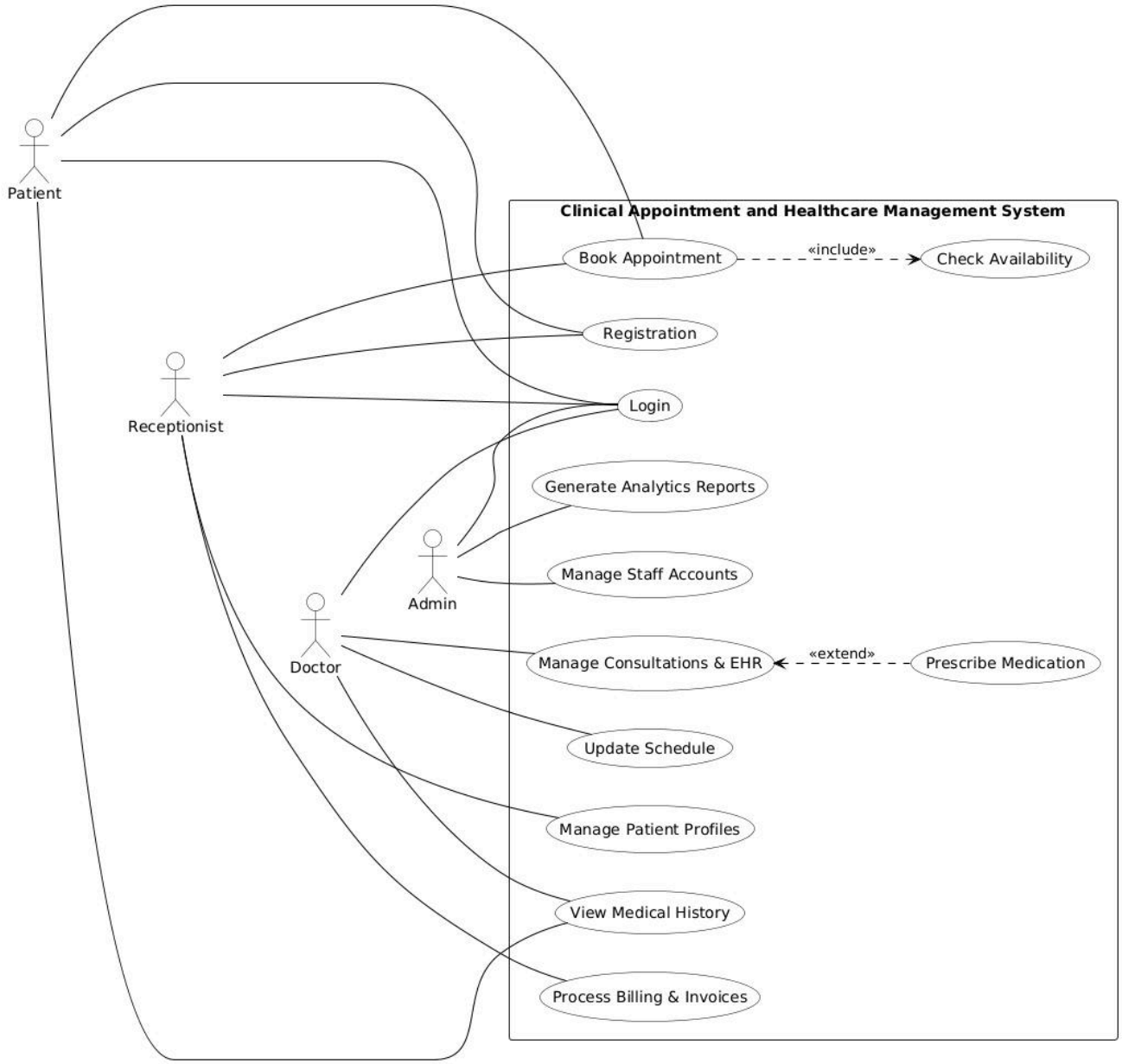
- **Algorithmic Scheduling:** A real-time booking interface equipped with double-booking prevention and temporal validation (preventing past-date bookings).
- **Dynamic Doctor Schedules:** Functionality for medical staff to view daily rosters and manage availability by canceling or blocking out specific time slots.
- *Architectural Note:* Employs the **Facade Design Pattern** to abstract the complexity of external subsystems, simulating automated SMS and email notifications upon appointment state changes.

IV. Financials & Analytics

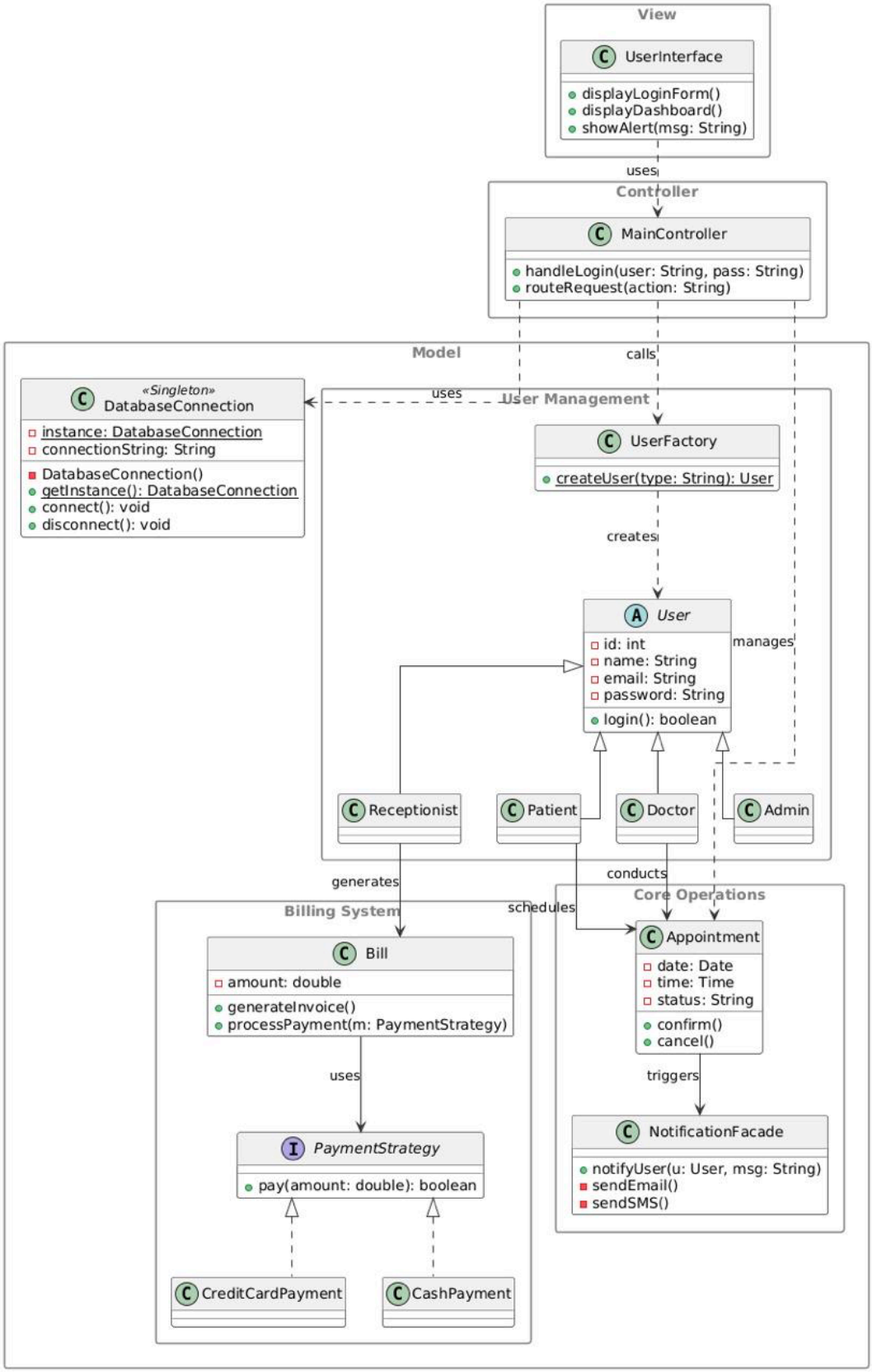
- **Automated Billing:** Generation of detailed medical invoices tied to specific patient consultations and appointments.
- **Administrative Dashboard:** Generation of analytics reports to track clinic revenue, appointment volumes, and operational statistics.
- *Architectural Note:* Uses the **Strategy Design Pattern** to abstract and interchange different payment processing algorithms (e.g., cash versus card transactions) at runtime.

Models:

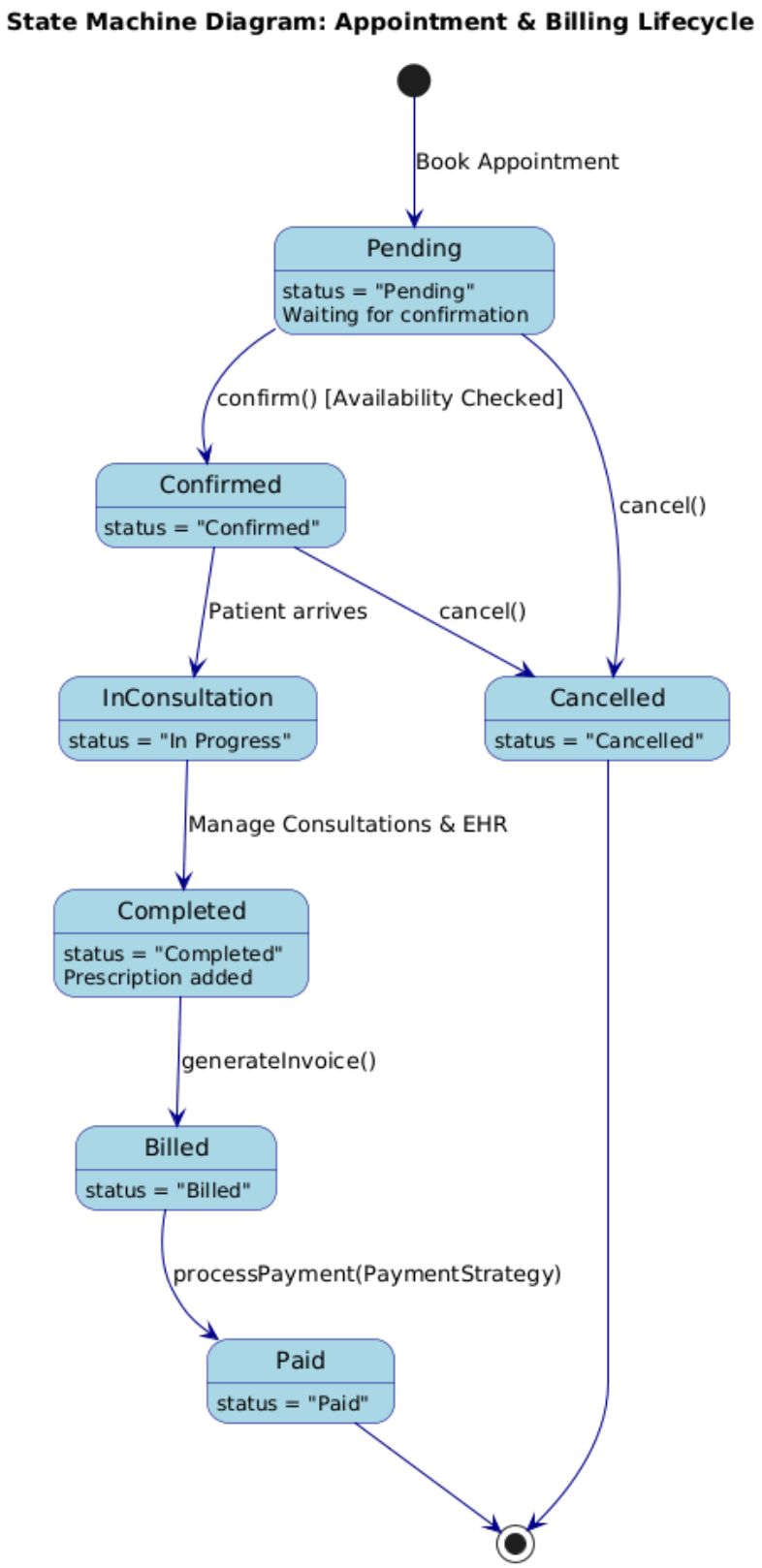
Use Case Diagram:



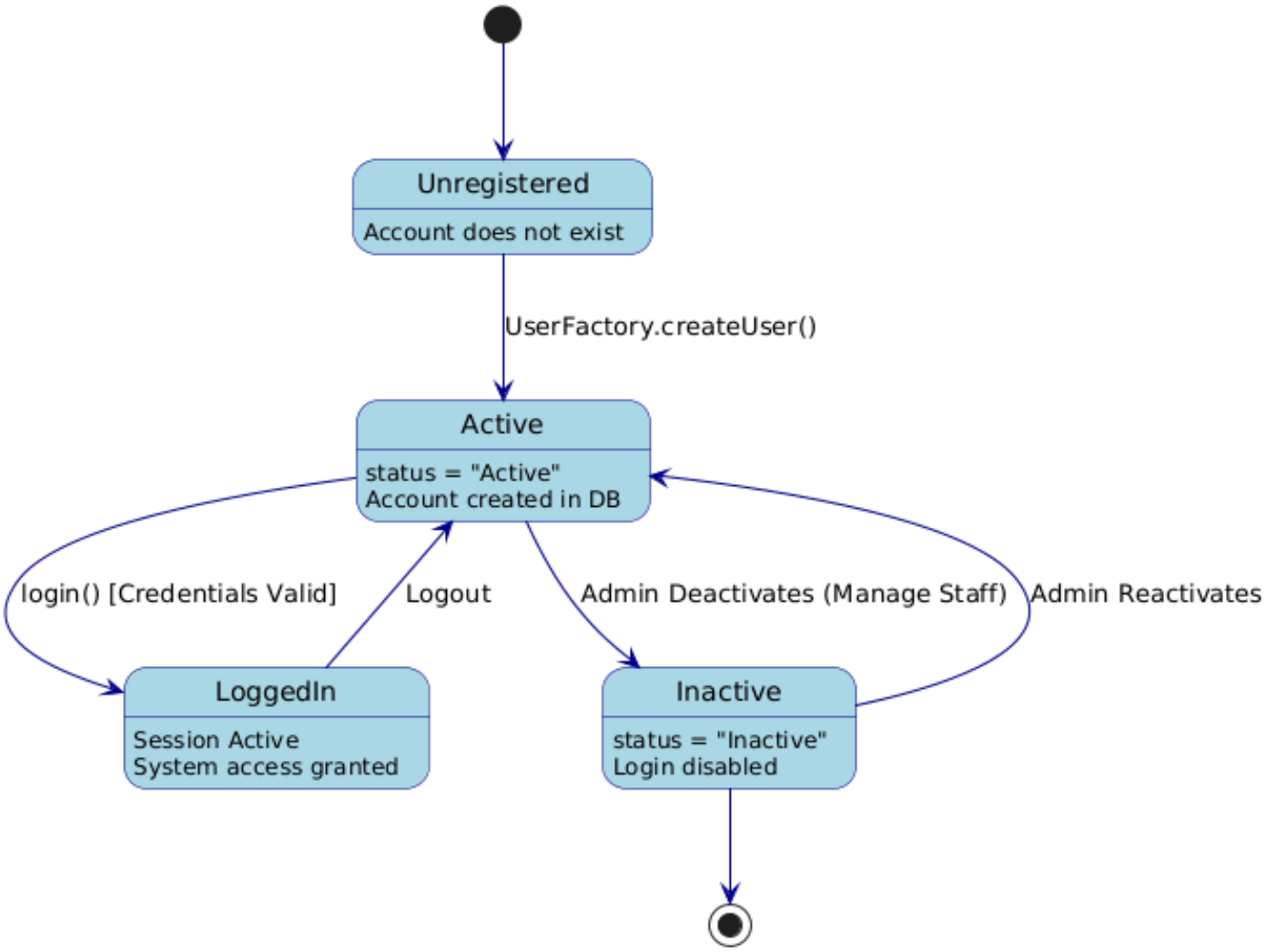
Class Diagram:



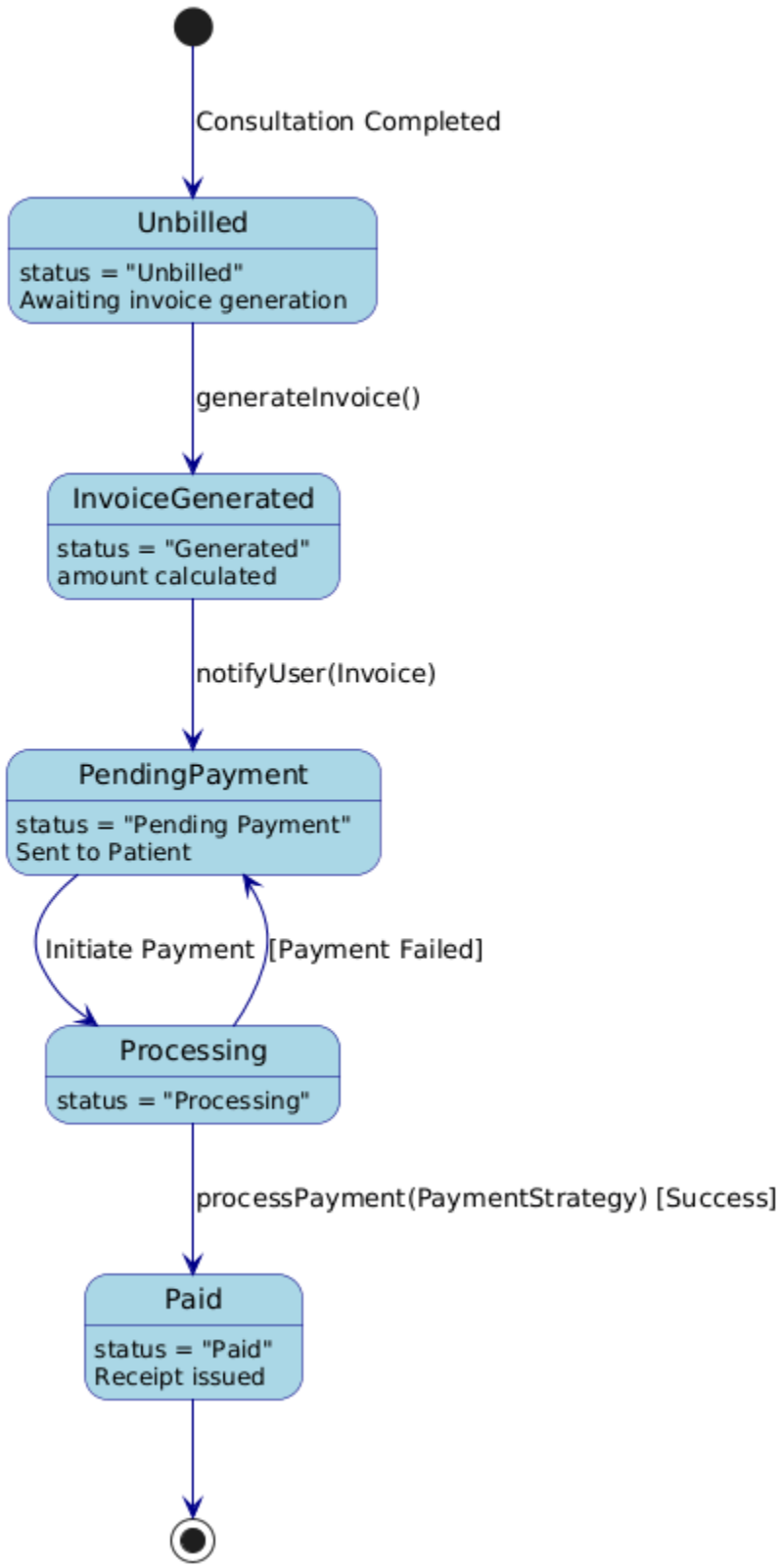
State Diagram:



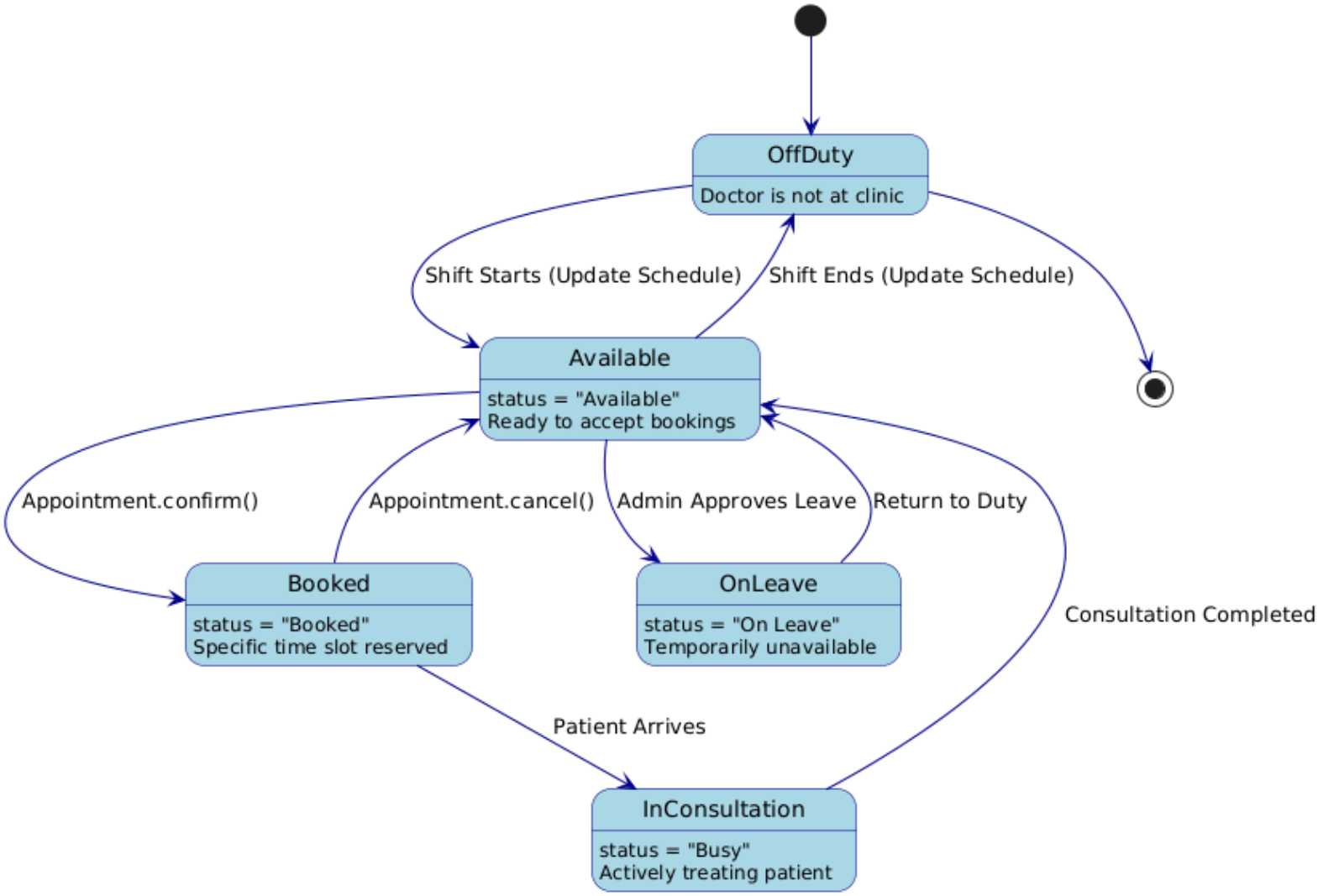
State Machine Diagram: User Account Lifecycle



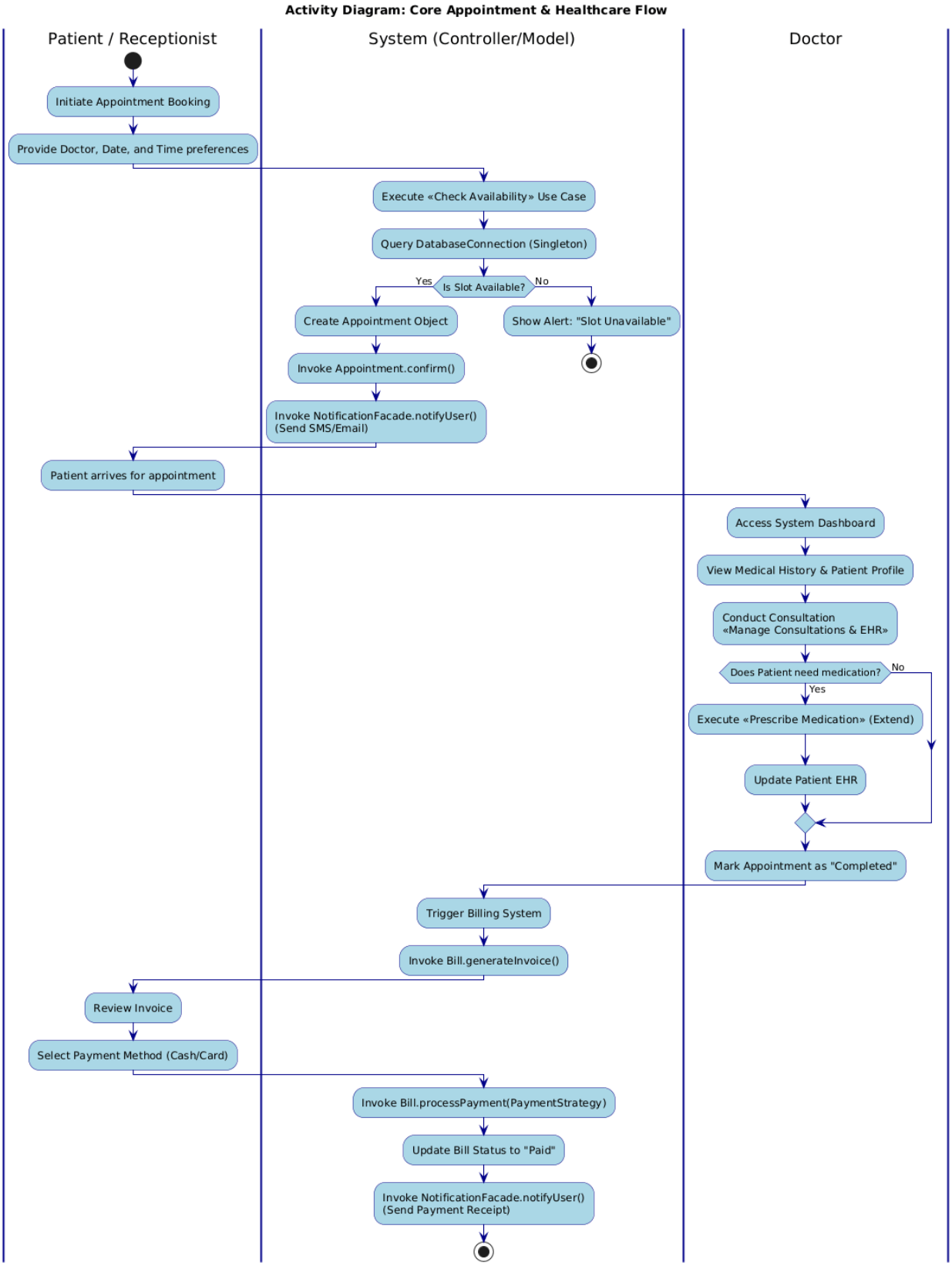
State Machine Diagram: Billing & Invoice Lifecycle



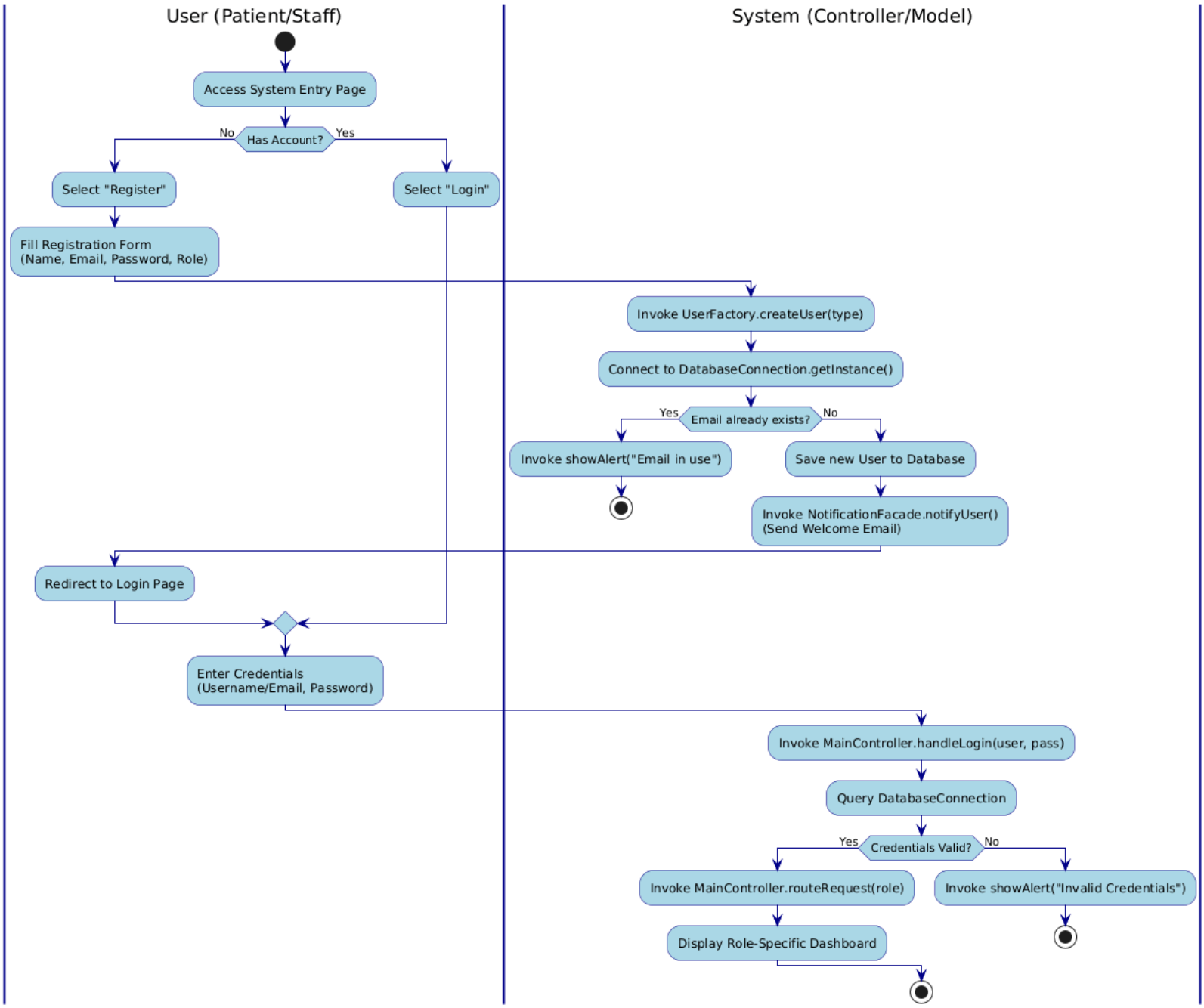
State Machine Diagram: Doctor Availability & Schedule



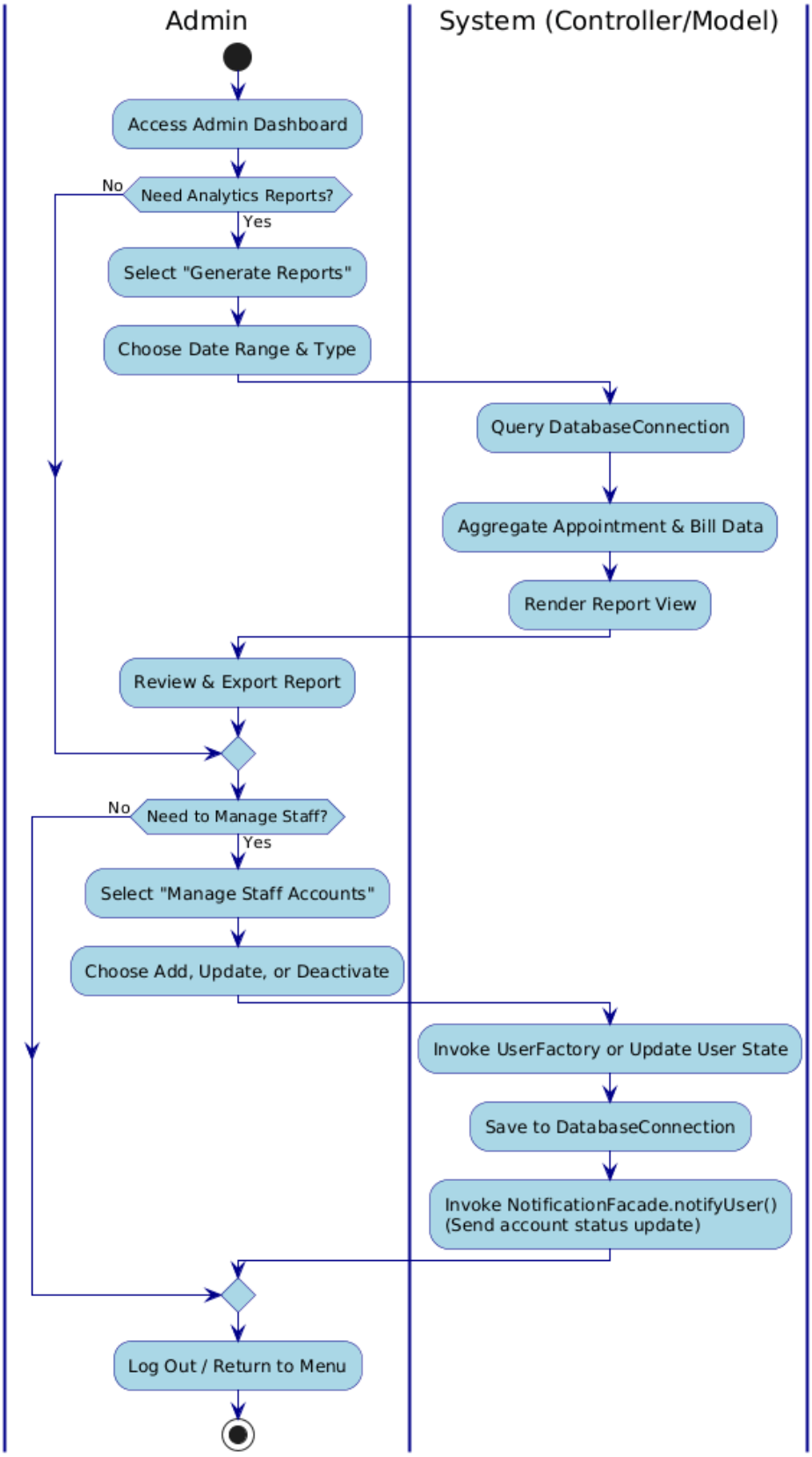
Activity Diagrams:



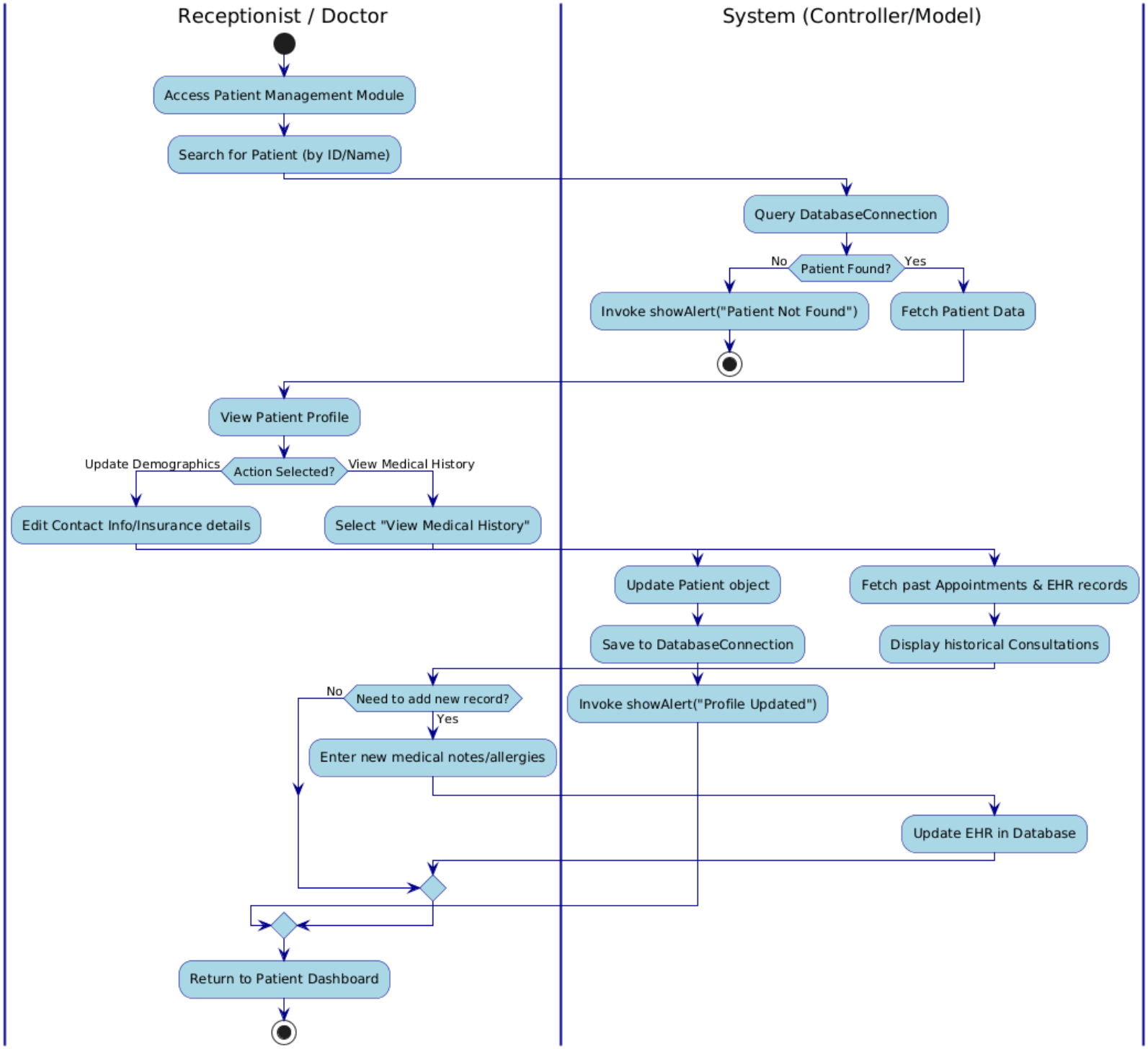
Activity Diagram: Registration & Login Flow



Activity Diagram: Admin Analytics & Staff Management



Activity Diagram: Patient Profile & Medical History Management



Design Principles, and Design Patterns:

MVC Architecture used? Yes

Design Principles

- **Single Responsibility Principle (SRP):** Classes are designed to handle one specific functional area. For example, the Bill class is strictly responsible for invoice calculations and payment tracking, while the Appointment class solely manages scheduling data and status updates.
- **Open/Closed Principle (OCP):** The payment module is open for extension but closed for modification. By relying on the PaymentStrategy interface, the system can support new payment types (such as UPI or Insurance) in the future without requiring changes to the core Bill class.
- **Dependency Inversion Principle (DIP):** High-level business logic modules do not depend on low-level utility classes. Instead of the Bill class depending directly on concrete implementations like CreditCardPayment, it depends entirely on the PaymentStrategy abstraction.
- **Encapsulate What Varies:** The process of instantiating different user roles requires conditional logic that varies based on input. This creation logic is encapsulated entirely within the UserFactory rather than being scattered across the MainController.

Design Patterns

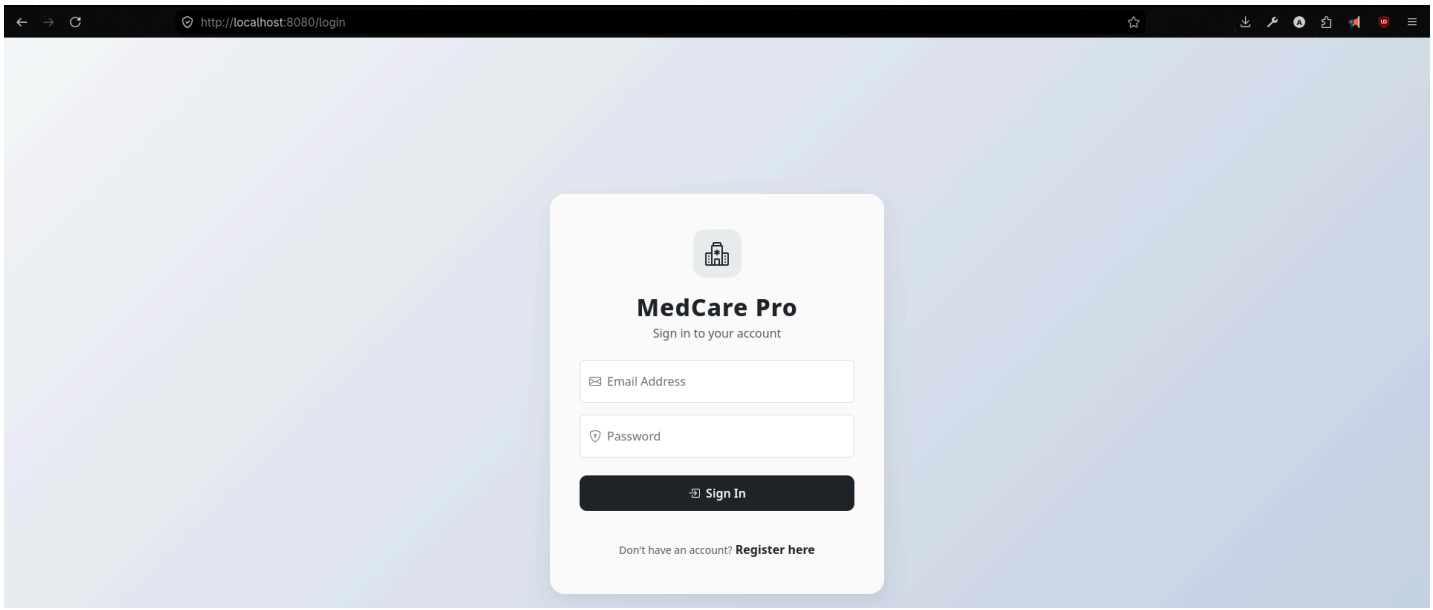
- **Singleton Pattern:** Applied to the DatabaseConnection class. This ensures that only one instance of the database connection exists throughout the application lifecycle, which manages resources efficiently and prevents concurrent connection leaks.
- **Factory Pattern:** Applied via the UserFactory class. It handles the creation of various user entity objects (Patient, Doctor, Receptionist, Admin). This centralizes object creation logic and keeps the main application controllers clean of direct instantiation code.
- **Strategy Pattern:** Applied using the PaymentStrategy interface and its concrete implementations (CreditCardPayment, CashPayment). This allows the Bill class to process payments interchangeably and dynamically switch between different payment algorithms at runtime.
- **Facade Pattern:** Applied to the NotificationFacade. It provides a unified, simplified interface for triggering alerts. It abstracts the underlying complexities of the individual SMS and Email subsystems, allowing the core operations to send notifications with a single method call.

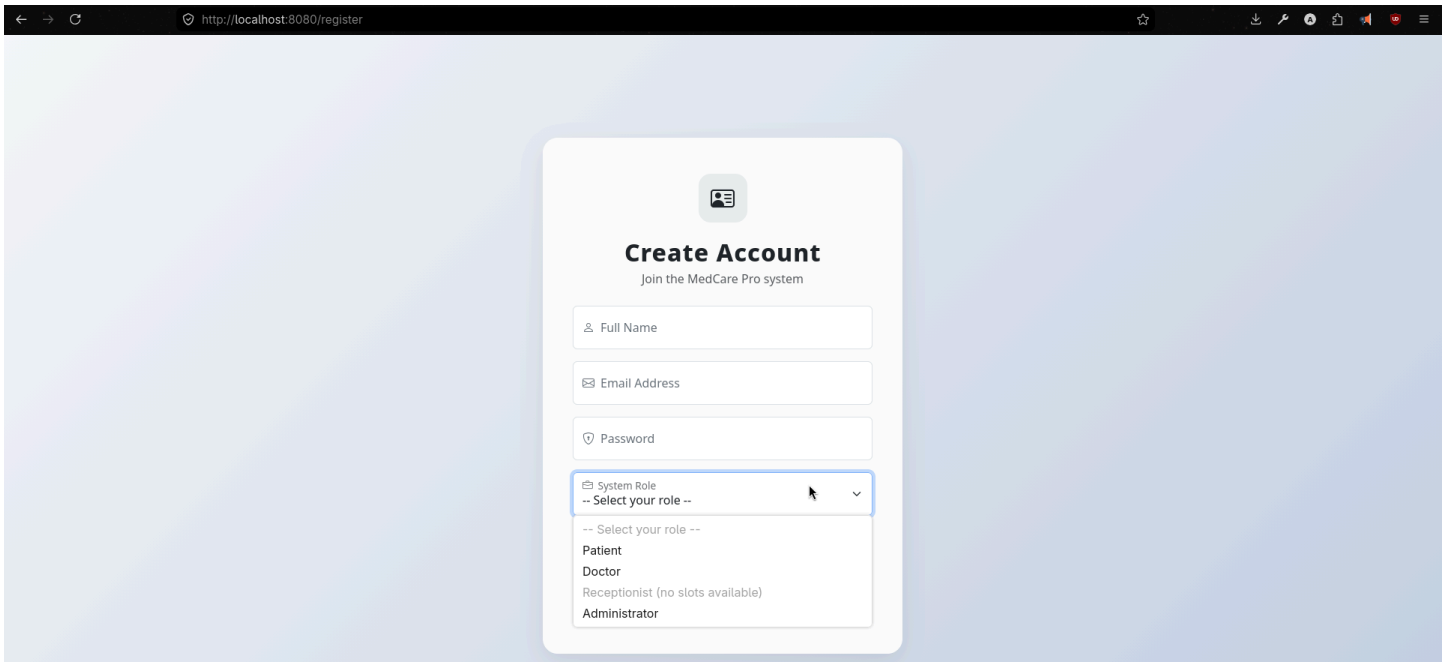
Github link to the Codebase: (repository should be public)

https://github.com/Prathameshshenoy/OOAD-Healthcare-Management.git

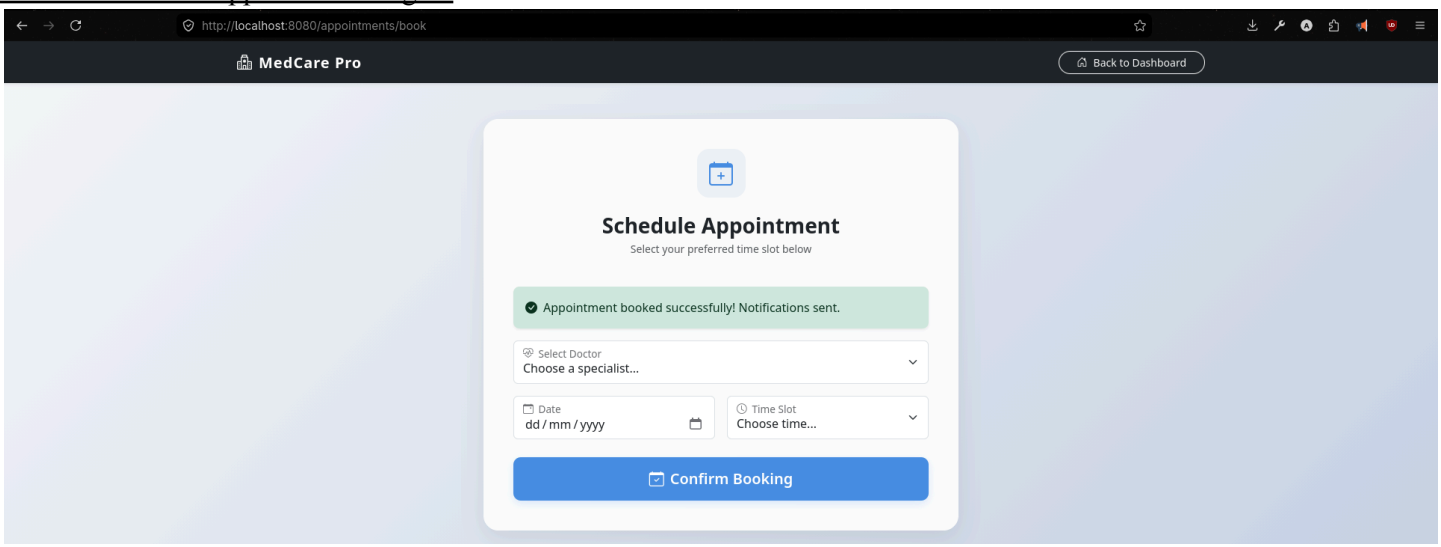
Screenshots:

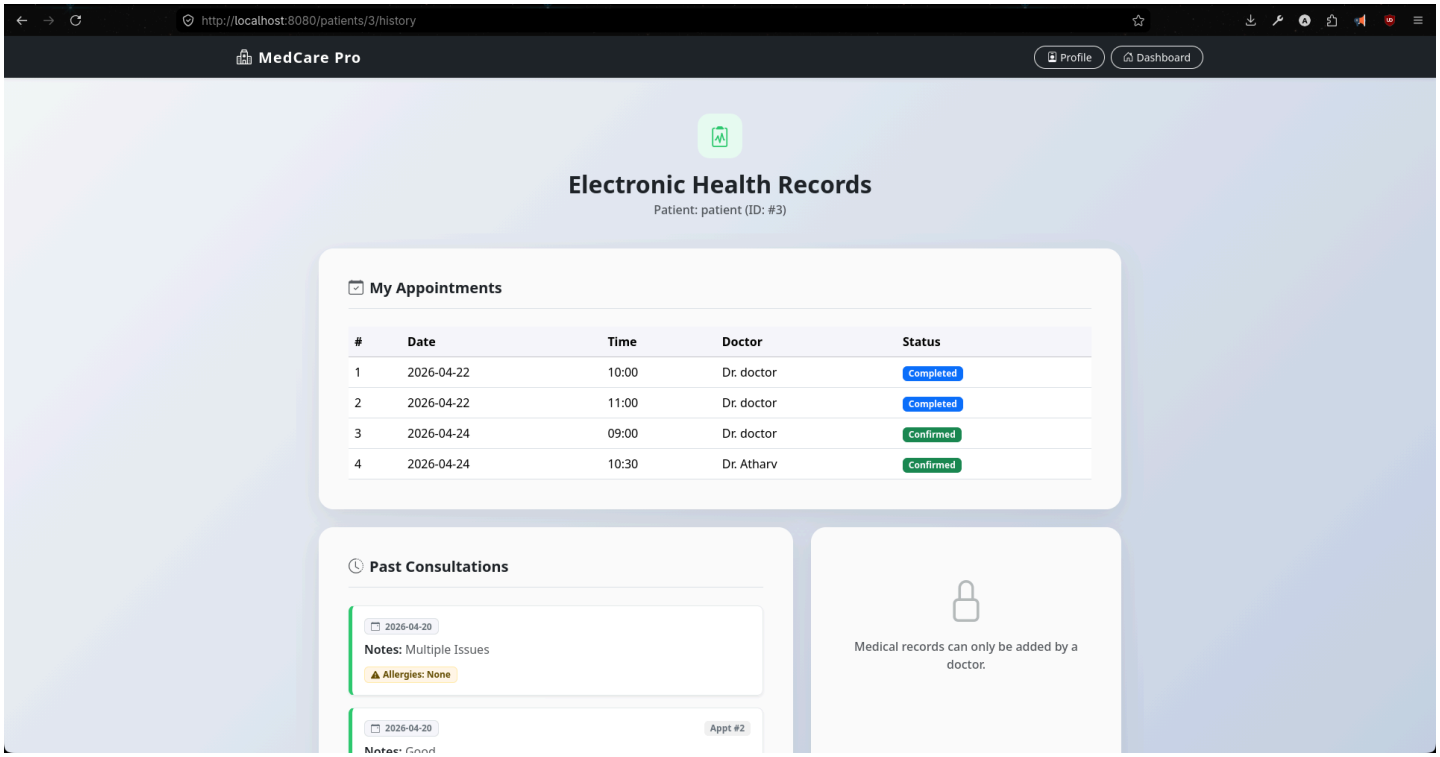
Module I: Authentication & Access Control



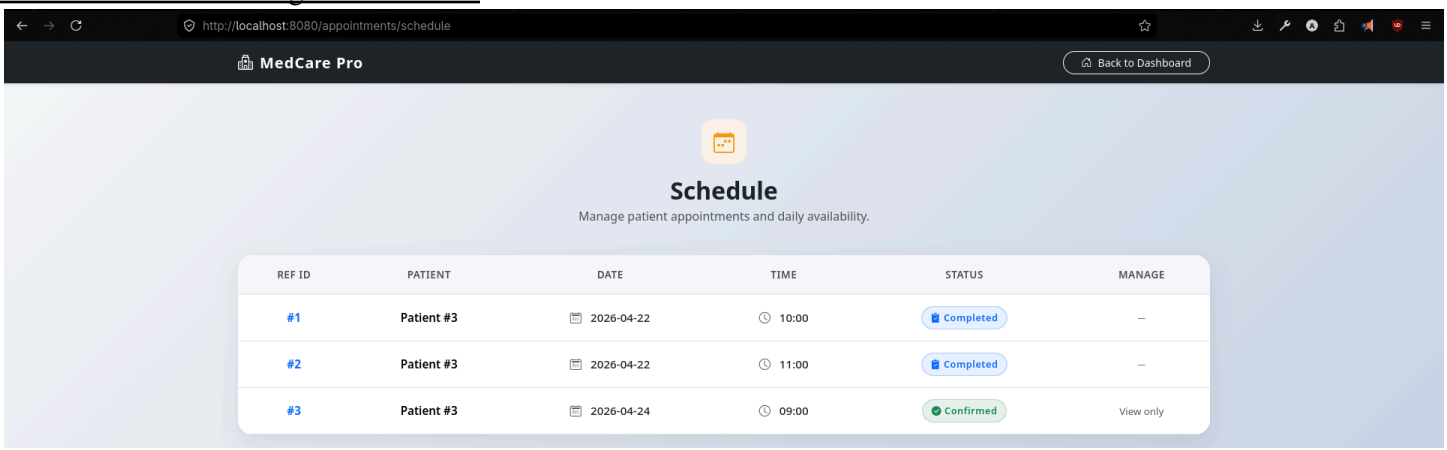


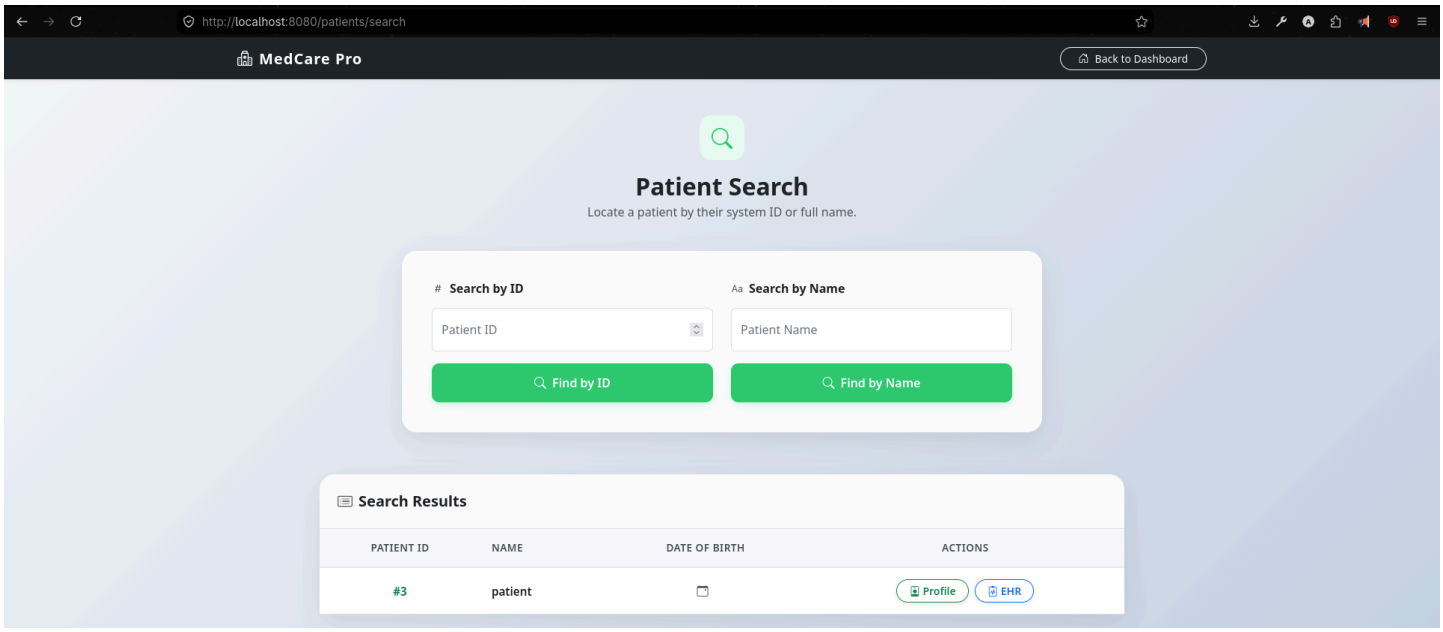
Module III: Core Appointment Engine



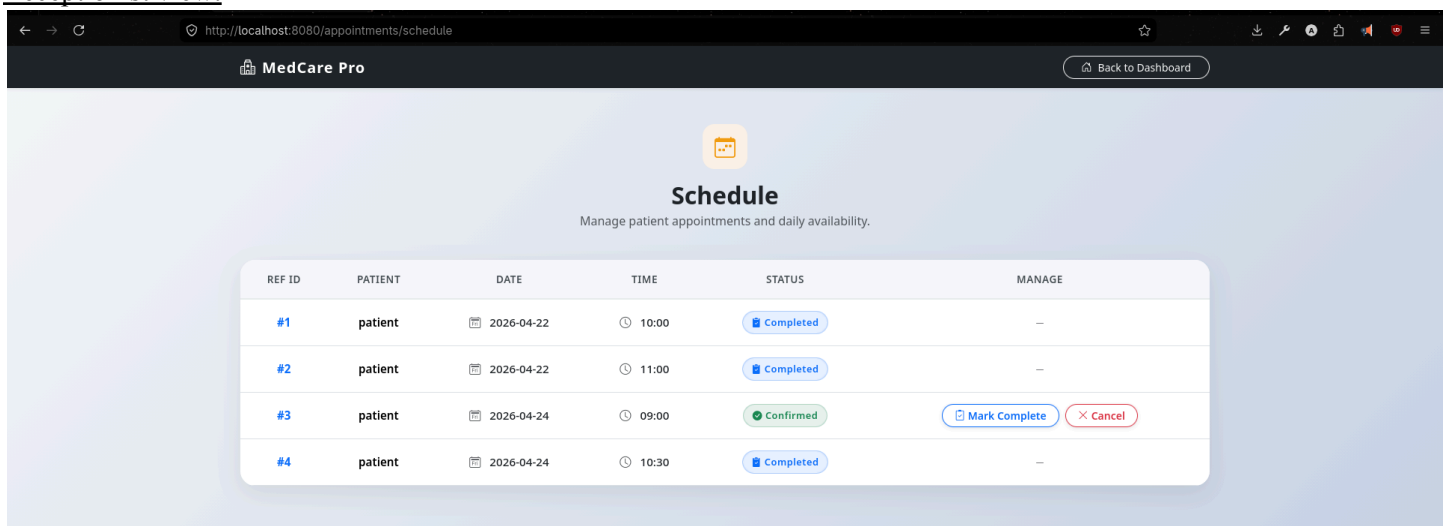


Module II: Patient Management & EHR:

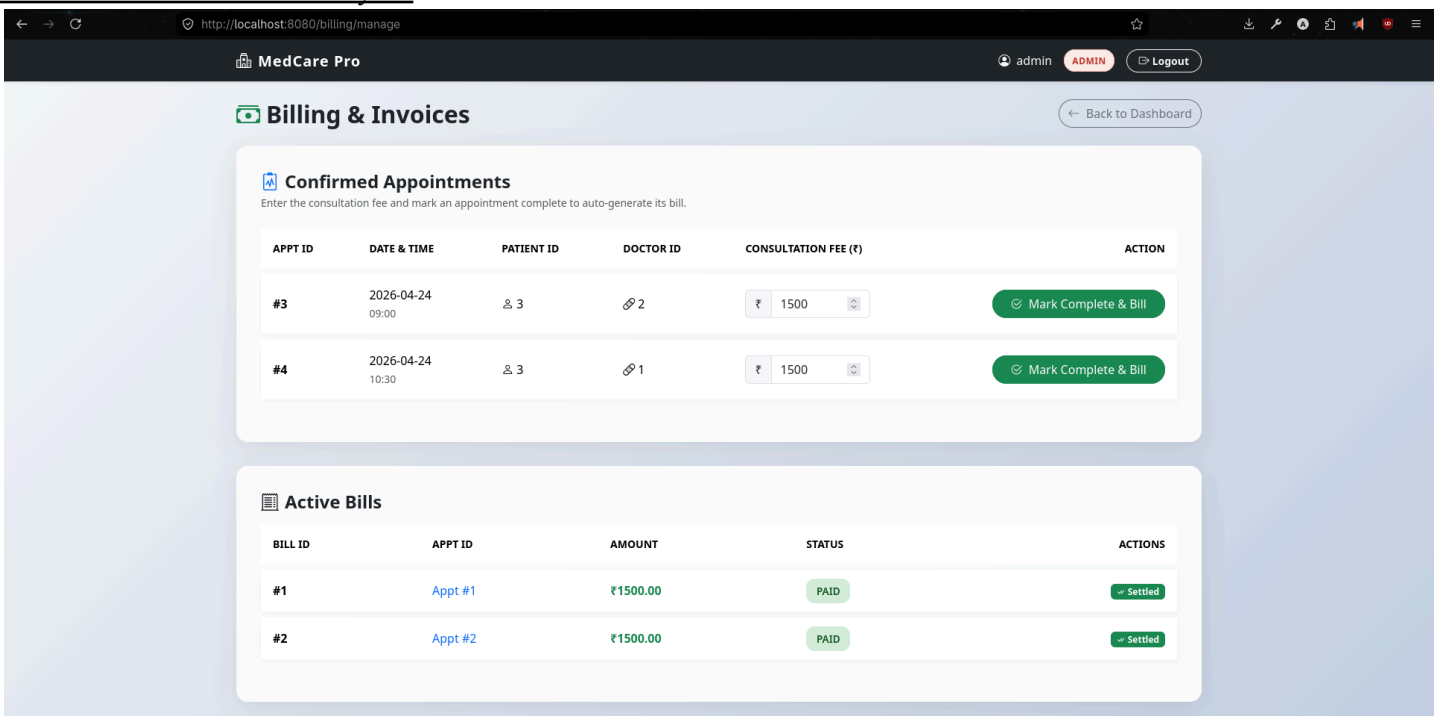


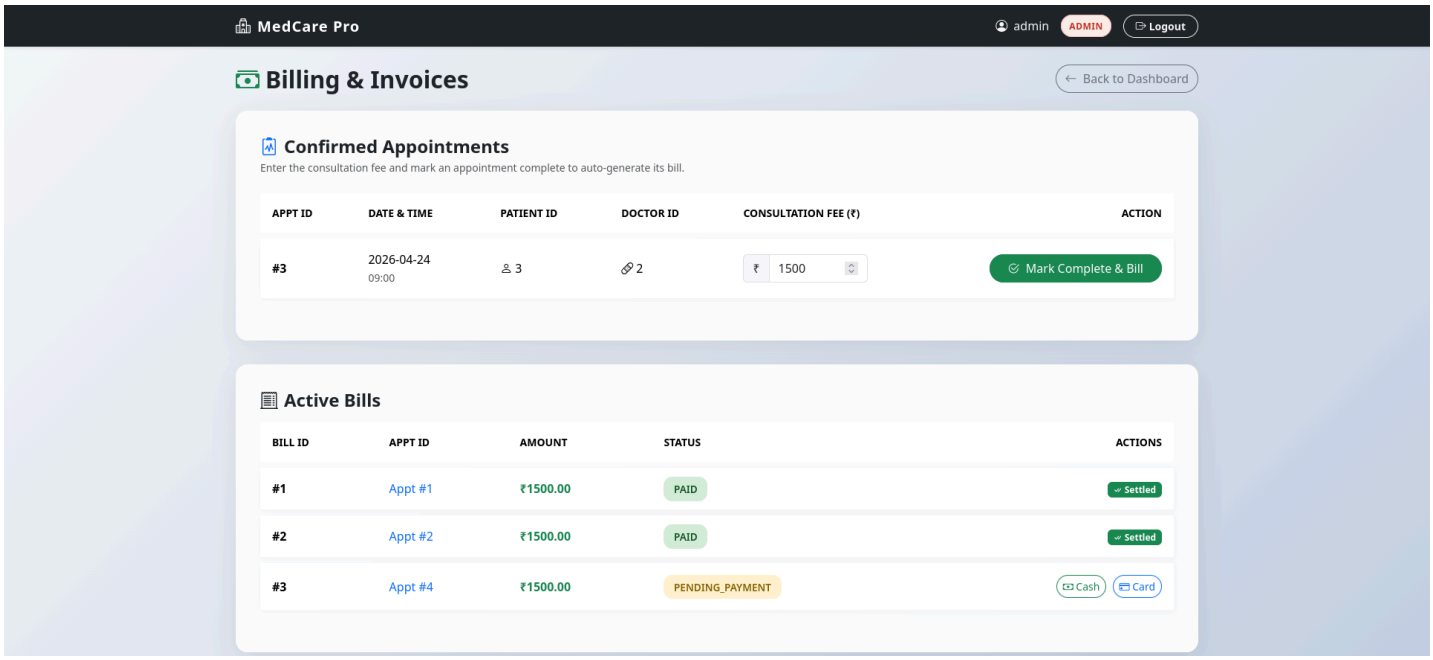


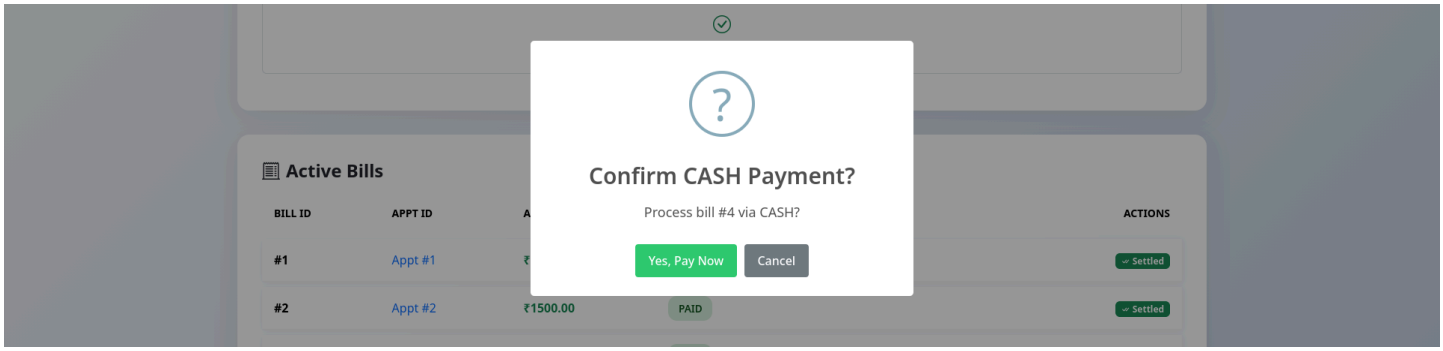
Receptionist view:

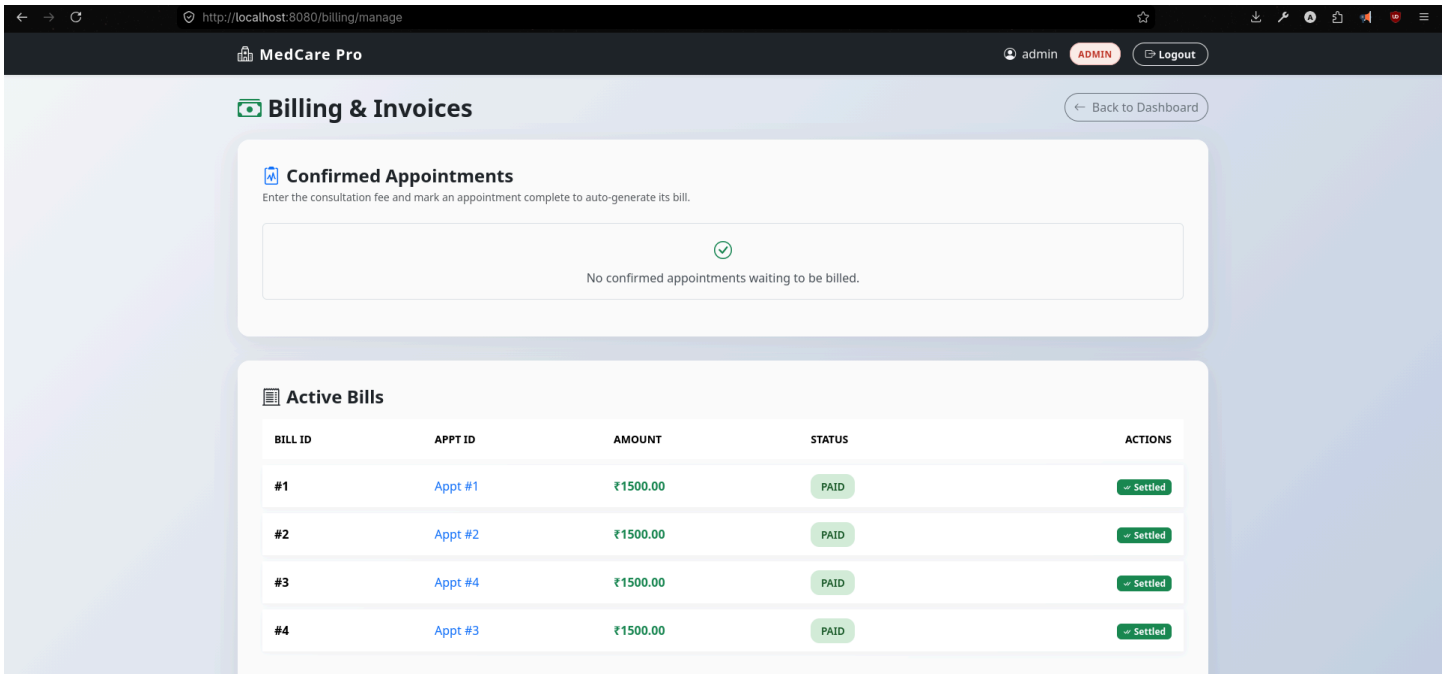


Module IV: Financials & Analytics

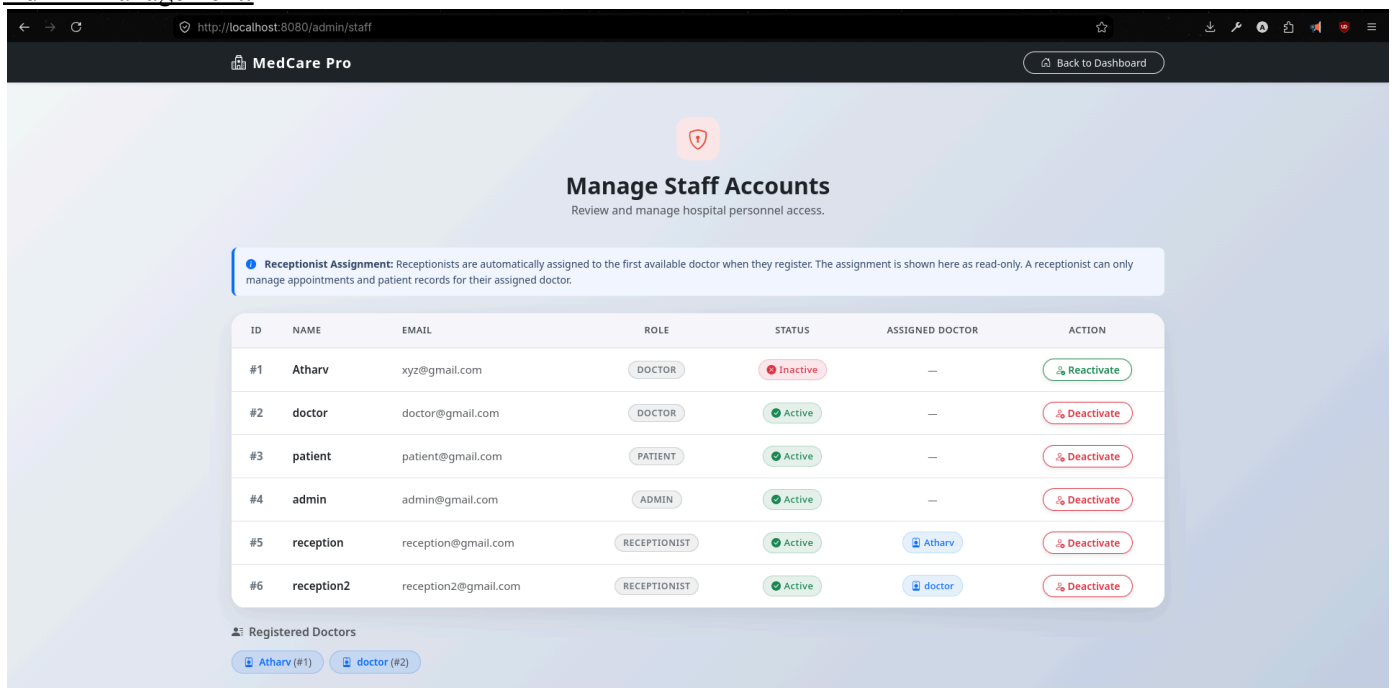








Admin Management:



H2 Database:

http://localhost:8080/h2-console/login.do?jsessionid=2b7b665bd8a6db1a902ba18edf33006e

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:file:./data/healthcare_db
- APPOINTMENTS
- BILLS
- MEDICAL_RECORDS
- PATIENTS
- USERS
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM APPOINTMENTS
```

SELECT * FROM APPOINTMENTS;

ID	APPOINTMENT_DATE	APPOINTMENT_TIME	DOCTOR_ID	PATIENT_ID	STATUS
1	2026-04-22	10:00:00	2	3	Completed
2	2026-04-22	11:00:00	2	3	Completed
3	2026-04-24	09:00:00	2	3	Confirmed
4	2026-04-24	10:30:00	1	3	Completed

(4 rows, 2 ms)

Edit

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:file:./data/healthcare_db
- APPOINTMENTS
- BILLS
- MEDICAL_RECORDS
- PATIENTS
- USERS
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM BILLS
```

SELECT * FROM BILLS;

ID	AMOUNT	APPOINTMENT_ID	STATUS
1	1500.0	1	PAID
2	1500.0	2	PAID
3	1500.0	4	PAID

(3 rows, 1 ms)

http://localhost:8080/h2-console/login.do?jsessionid=2b7b665bd8a6db1a902ba18edf33006e

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:file:./data/healthcare_db
- APPOINTMENTS
- BILLS
- MEDICAL_RECORDS
- PATIENTS
- USERS
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM MEDICAL_RECORDS
```

SELECT * FROM MEDICAL_RECORDS;

RECORD_ID	ALLERGIES	CONSULTATION_NOTES	LINKED_APPOINTMENT_ID	RECORD_DATE	PATIENT_ID
1	None	Multiple Issues	*null*	2026-04-20	3
2	None	Good	2	2026-04-20	3

(2 rows, 0 ms)

Edit

http://localhost:8080/h2-console/login.do?jsessionid=2b7b665bd8a6db1a902ba18edf33006e

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:file:./data/healthcare_db
- APPOINTMENTS
- BILLS
- MEDICAL_RECORDS
- PATIENTS
- USERS
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM PATIENTS
```

SELECT * FROM PATIENTS;

PATIENT_ID	CONTACT_INFO	DATE_OF_BIRTH	INSURANCE_INFO	NAME
3	*null*	*null*	*null*	patient

(1 row, 0 ms)

Edit

http://localhost:8080/h2-console/login.do?jsessionid=2b7b665bd8a6db1a902ba18edf33006e

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:file:./data/healthcare_db
- APPOINTMENTS
- BILLS
- MEDICAL_RECORDS
- PATIENTS
- USERS
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM USERS
```

SELECT * FROM USERS;

DTYPE	ID	ASSIGNED_DOCTOR_ID	EMAIL	NAME	PASSWORD	ROLE	STATUS
DOCTOR	1	*null*	xyz@gmail.com	Atharv	atharvrawal	DOCTOR	Inactive
DOCTOR	2	*null*	doctor@gmail.com	doctor	doctor	DOCTOR	Active
PATIENT	3	*null*	patient@gmail.com	patient	patient	PATIENT	Active
ADMIN	4	*null*	admin@gmail.com	admin	admin	ADMIN	Active
RECEPTIONIST	5	1	reception@gmail.com	reception	reception	RECEPTIONIST	Active
RECEPTIONIST	6	2	reception2@gmail.com	reception2	reception2	RECEPTIONIST	Active

(6 rows, 1 ms)

Individual contributions of the team members:

Name	Module worked on
Arkul Prathamesh Shenoy	III. Core Appointment Engine
Arjun MS	IV. Financials & Analytics
Aryan Arun Hiremath	I. Authentication & Access Control
Atharv Rawal	III. Patient Management & Electronic Health Records (EHR)